Lab Assignment 4

Title:

To execute SQL queries using various **data retrieval clauses** such as `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, and `DISTINCT`.

Objective:

To understand how to retrieve and organize data from database tables using different SQL clauses effectively.

Problem Statement:

Using the previously created database tables (`Customer`, `Itemss`, `Orders`, and `Order_Details`), execute SQL queries to perform various data retrieval operations involving filtering, sorting, grouping, and aggregation.

Sample Data Setup (if tables not already created):

Use the same tables created in **Lab Assignment 3** or run the following simplified inserts for testing:

```
INSERT INTO Customer VALUES (1, 'Rahul Sharma', 'Aurangabad',
'9876543210');
INSERT INTO Customer VALUES (2, 'Sneha Patil', 'Pune', '9823456789');
INSERT INTO Customer VALUES (3, 'Amit Deshmukh', 'Mumbai', '9991122334');

INSERT INTO Items VALUES (101, 'Keyboard', 850.00, 'Computer');
INSERT INTO Items VALUES (102, 'Mouse', 450.00, 'Computer');
INSERT INTO Items VALUES (103, 'Monitor', 12500.00, 'Display');
INSERT INTO Items VALUES (104, 'Speaker', 2200.00, 'Audio');

INSERT INTO Orders VALUES (1001, 1, '2025-10-10');
INSERT INTO Orders VALUES (1002, 2, '2025-10-11');
INSERT INTO Orders VALUES (1003, 3, '2025-10-12');

INSERT INTO Order_Details VALUES (1001, 101, 2, 5.0);
INSERT INTO Order_Details VALUES (1001, 102, 1, 0);
INSERT INTO Order_Details VALUES (1002, 103, 1, 10.0);
INSERT INTO Order_Details VALUES (1003, 104, 2, 0);
```

Queries:

1. Retrieve all customers from Pune

```
SELECT * FROM Customer
WHERE City = 'Pune';
```

2. Display all items whose price is greater than 1000 and less than 5000

```
SELECT ItemId, ItemName, Price
FROM Items
WHERE Price BETWEEN 1000 AND 5000;
```

3. Display all unique categories of items

```
SELECT DISTINCT Category
FROM Items;
```

4. Display all items sorted by price in descending order

```
SELECT ItemName, Price
FROM Items
ORDER BY Price DESC;
```

5. Find total quantity sold of each item

```
SELECT i.ItemName, SUM(od.Quantity) AS TotalQtySold
FROM Items i
JOIN Order_Details od ON i.ItemId = od.ItemId
GROUP BY i.ItemName;
```

6. Display total sales amount for each customer

```
SELECT c.CustName, SUM(i.Price * od.Quantity - od.Discount) AS
TotalPurchase
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
JOIN Order_Details od ON o.OrderId = od.OrderId
JOIN Items i ON od.ItemId = i.ItemId
GROUP BY c.CustName;
```

7. Display customers whose total purchase exceeds ₹5,000

```
SELECT c.CustName, SUM(i.Price * od.Quantity - od.Discount) AS
TotalPurchase
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
JOIN Order_Details od ON o.OrderId = od.OrderId
JOIN Items i ON od.ItemId = i.ItemId
GROUP BY c.CustName
HAVING SUM(i.Price * od.Quantity - od.Discount) > 5000;
```

8. Retrieve order details where discount is not zero

```
SELECT o.OrderId, c.CustName, i.ItemName, od.Discount
FROM Orders o
JOIN Customer c ON o.CustId = c.CustId
JOIN Order_Details od ON o.OrderId = od.OrderId
JOIN Items i ON od.ItemId = i.ItemId
WHERE od.Discount <> 0;
```

9. Display items along with their category, sorted by category and price

```
SELECT ItemName, Category, Price
FROM Items
ORDER BY Category ASC, Price DESC;
```

10. Display number of orders placed by each customer

```
SELECT c.CustName, COUNT(o.OrderId) AS TotalOrders
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
GROUP BY c.CustName;
```

Result:

Various SQL clauses such as **WHERE, ORDER BY, DISTINCT, GROUP BY, and HAVING** were successfully demonstrated to retrieve and analyze data from multiple related tables.